

Introduction to Data Analysis

Week 3. Data Collection and File Handling

Jin Hee Yoon

2026

Sejong University

Week 3 Overview

2주차에서 데이터의 유형과 구조를 다루었다면, 3주차에서는 외부에 저장된 데이터를 안전하게 불러오고 다시 저장하는 과정을 다룬다.

- **Week 3-1:** file format, path, delimiter, header, encoding 등 파일을 다룰 때 알아야 하는 핵심 개념을 설명한다.
- **Week 3-2:** TXT, CSV, Excel 파일을 Python으로 만들고 불러오며, 주요 option을 적용하고 처리된 데이터를 저장한다.

어떤 설정으로 파일을 읽었는지 기록하고, 불러온 결과가 원본의 의미와 일치하는지 확인하는 것이 파일을 읽는 명령보다 더 중요하다.

Topics of Week 3

	주제	Week 3-1 (개념)	Week 3-2 (실습)
1	Data collection	출처와 이력	출처 정보 기록
2	File format	TXT, CSV, Excel	읽기와 쓰기
3	File path	현재 폴더와 path	os, Path
4	Delimiter와 header	열 구분과 열 이름 설정	sep, header, names
5	Encoding	문자를 저장하는 방식의 차이	UTF-8, CP949
6	Loading options	원하는 열과 타입만 골라서 읽기	usecols, dtype, parse_dates
7	Validation	불러온 결과가 맞는지 점검	shape, info(), 범위 확인
8	Saving workflow	raw와 processed 파일 분리	CSV/Excel 저장 후 재확인

Week 3-1. Data Collection and File Handling

Why File Handling Matters

- CSV 데이터가 comma로 구분된 파일이다. 예를 들어, semicolon 으로 구분하면 여러 열이 하나의 열로 뭉쳐서 들어온다.
- header가 없는 파일을 그냥 읽으면 첫 번째 데이터 행이 column name 자리로 올라간다.
- encoding이 맞지 않으면 한글이 깨지거나 파일 자체가 열리지 않는다.(ASCII 범위 밖의 문자의 경우 파일을 저장할 때 사용한 번역 규칙과 파일을 읽어 들일 때 적용한 번역 규칙이 다르면 오류생김)
- 날짜 열이 문자로 읽히면 두 날짜 사이의 차이를 계산하거나 월별로 묶는 작업을 할 수 없다.

참고) ASCII(American Standard Code for Information Interchange, 미국 정보 교환 표준 부호): 영문 알파벳을 사용하는 대표적인 초기 문자 인코딩 표준. 영문알파벳, 숫자, 특수기호 128개

파일을 불러오는 것은 단순한 열기 작업이 아니라, 원본 데이터의 구조와 의미를 **Python DataFrame**으로 정확히 옮기는 과정이다.

Topic 1. Data Collection

데이터는 다양한 곳에서 가져올 수 있다.

Source	예	특징
Direct measurement	sensor, experiment	측정 조건의 기록이 중요
Survey	questionnaire	문항과 응답 방식 확인
Administrative record	학교, 병원, 정부	정의와 갱신 주기 확인
Web / API	website, open API	호출 시점과 구조 확인
File	TXT, CSV, Excel	format과 option 확인

- API(Application Programming Interface): 서로 다른 소프트웨어 프로그램이 규약에 따라 데이터를 주고받고 소통할 수 있도록 연결해 주는 매개체

파일을 받았다고 분석 준비가 끝난 것은 아니다. 어디서 왔는지, 언제 수집됐는지, 한 행이 무엇인지, 각 열의 의미가 무엇인지를 알아야 파일을 제대로 읽을 수 있다.

Topic 1. Data Provenance

Data provenance는 데이터가 어디서 왔고, 어떤 과정을 거쳐 지금 이 모습이 되었는지를 기록한 정보이다.

항목	질문	예
Source	어디에서 얻었는가?	학교 행정시스템
Collected at	언제 수집했는가?	2026-03-31
Observation unit	한 행은 무엇인가?	학생 한 명
Variable definition	각 열은 무엇인가?	StudyTime = 주당 학습 분
Version	어느 시점의 파일인가?	v2026_03
가공 여부	이미 손이 간 파일인가?	중복 제거 전 raw file

file 이름이 같은 `score.csv`라도 언제 수집했고 어떤 처리를 거쳤는지에 따라 전혀 다른 데이터일 수 있다.

Topic 2. Common File Formats

Format	구조	장점	Python
TXT	plain text	단순하고 범용적	<code>open()</code>
CSV	delimiter 기반 table	가볍고 호환성 높음	<code>pd.read_csv()</code>
Excel	spreadsheet	여러 sheet와 서식	<code>pd.read_excel()</code>
JSON	key-value / nested	web/API에 흔함	<code>pd.read_json()</code>

- delimiter: 데이터를 나눌 때 쓰는 구분 기호
- JSON(JavaScript Object Notation): 데이터를 저장하거나 네트워크를 통해 전송할 때 사용하는 경량의 텍스트 기반 데이터 교환 형식

확장자는 file format을 짐작하게 해 주는 단서일 뿐, 내부 구조까지 보장하지는 않는다. 이름이 CSV라도 열 구분 기호가 comma가 아니라 semicolon이나 tab일 수 있다.

Topic 2. TXT Is Not One Single Structure

TXT는 텍스트로만 이루어진 파일이라는 뜻일 뿐, 내부가 어떤 구조인지는 따로 확인해야 한다.

형태	예	읽는 방법
문장형	로그, 메모, 문서	<code>open().read()</code>
공백 구분	201 85 20	<code>read_csv(sep=...)</code>
Tab 구분	TSV(Tab-Separated Values)	<code>read_csv(sep="\t")</code>
고정 폭	자릿수별 의미	전용 option 또는 parsing

- TSV(Tab-Separated Values): 데이터를 테이블 형태로 저장할 때, 열(Column)과 열 사이를 탭 문자(`\t`)로 구분하는 텍스트 기반 데이터 교환 포맷
- 파싱(Parsing, 구문 분석): 일련의 문자열이나 텍스트 데이터를 정해진 문법(규칙)에 따라 분해하고 해석하여, 컴퓨터 프로그램이 다루기 쉬운 구조화된 데이터(객체, 트리 구조 등)로 변환하는 과정

TXT 파일을 읽기 전에 **한 줄이 무엇을 나타내는지, 값과 값 사이를 무엇으로 구분하는지**를 먼저 확인한다.

Topic 2. What Makes a CSV File?

CSV는 comma로 값을 구분하여 저장하는 텍스트 파일이다.

ID	Store	Product	Revenue
1	A	Coffee	80000
2	A	Tea	45000
3	B	Coffee	100000

파일을 텍스트 편집기로 열면 실제로 다음과 같이 저장되어 있다.

```
ID,Store,Product,Revenue
1,A,Coffee,80000
2,A,Tea,45000
3,B,Coffee,100000
```

CSV에는 Excel의 cell 서식이나 색상 정보가 없다. 모든 값이 텍스트로 저장되어 있고, 파일을 불러올 때 pandas가 각 열의 type을 스스로 판단한다.

Topic 2. CSV Can Be Tricky

CSV라는 이름만으로 내부 구조가 모두 정해지는 것은 아니다.

- delimiter(구분기호)가 comma인지, semicolon인지, tab인지 확인한다.
- 값 안에 delimiter가 포함된 경우 그 값이 quotation mark로 감싸져 있을 수 있다.
- 첫 줄이 header인지 실제 데이터인지 확인한다.
- 결측이 빈 칸인지, NA인지, -99 같은 약속된 숫자인지 확인한다.
- 한글 file은 UTF-8이 아니라 CP949로 저장된 경우도 있다.

참고) 한글이 포함된 텍스트/데이터 파일(CSV, TXT 등)이 전 세계 범용 표준인 UTF-8 대신, 과거부터 한국어 윈도우(Microsoft Windows) 시스템의 기본 규격으로 사용되어 온 CP949(확장 완성형) 방식으로 문자(바이트)를 변환해 저장되었을 수 있다는 의미. Python은 기본적으로 UTF-8로 가정함.

오류 없이 열렸다고 끝이 아니다. 행 수, 열 수, column name, type, 값 일부를 직접 확인해야 한다.

Topic 2. Excel Files

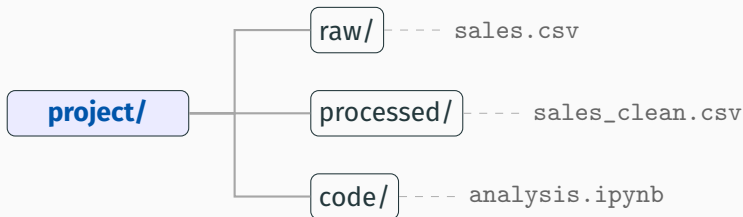
Excel은 사람이 보고 수정하기 편하지만, 분석용 파일로 쓸 때는 주의할 점이 있다.

- 하나의 파일 안에 여러 sheet가 있을 수 있으므로 어느 sheet를 읽을지 확인한다.
- 실제 data table 위에 제목, 설명, 빈 행이 끼어 있을 수 있다.
- 병합된 cell이나 배경색은 분석에 사용할 변수로 자동 변환되지 않는다.
- 숫자처럼 보이는 값이 text로 저장될 수 있다.
- formula가 들어 있는 cell은 계산된 결과값과 수식 자체를 구분해야 한다.

Excel 파일 하나에 보고서용 표와 분석용 표가 함께 들어 있는 경우가 많다. 어느 범위가 실제 data table인지 먼저 확인한다.

Topic 3. Working Directory

Working directory는 Python이 파일 경로의 시작점으로 삼는 폴더이다.



- working directory가 `project/`라면 `raw/sales.csv`라고만 써도 파일을 찾는다.
- 파일을 찾지 못하는 오류가 나면, 코드보다 먼저 현재 폴더와 파일 위치가 맞는지 확인한다.

Topic 3. Relative and Absolute Paths

File path는 두 가지 방식으로 표현할 수 있다.

Path	예	특징
Relative path	raw/sales.csv	working directory를 기준으로 표현
Absolute path	C:/project/raw/sales.csv	드라이브 루트부터 전체 경로를 직접 지정

- 내 PC 기준으로 작성한 absolute path는 다른 컴퓨터에서 그대로 쓰면 오류가 난다.
- 코드를 다른 사람과 공유할 때는 relative path를 쓰는 것이 안전하다.
- Windows의 경로 구분자 \는 Python에서 특수 문자로 해석될 수 있으니 주의한다.

project folder 하나를 정하고 그 안에서 relative path만 사용하면 어떤 컴퓨터에서도 코드가 그대로 동작한다.

Topic 3. Pathlib as a Safer Path Tool

Python의 `pathlib.Path`를 사용하면 Windows와 Mac/Linux 간의 path 표기 차이를 신경 쓰지 않아도 된다.

```
project / "raw" / "sales.csv"
```

- 문자열을 직접 이어 붙이는 대신 / 연산자로 folder와 file name을 연결한다.
- Windows와 macOS/Linux 사이에서 코드를 그대로 옮겨도 동작한다.

3-2 실습에서는 `Path("raw") / "sales.csv"`처럼 path를 만든다. 다음 모두 사용가능. 따옴표는 필수

```
Path('raw') / 'sales.csv'  
Path("raw") / "sales.csv"  
Path("raw/sales.csv")
```

Topic 4. Delimiter

Delimiter는 한 줄 안에서 값과 값 사이를 나누는 문자이다.

형식	예	sep
Comma-separated	1,A,85	","
Semicolon-separated	1;A;85	";"
Tab-separated	1\tA\t85	"\t"
Whitespace	1 A 85	상황에 따라 다름

Delimiter가 잘못 지정되었을 때 가장 흔한 증상은 여러 열이어야 할 데이터가 하나의 긴 열로 읽히는 것이다.

Topic 4. Header

Header는 각 열의 이름이 적힌 첫 번째 행이다.

Header가 있는 경우 ID, Major, Score

201, AI, 85

202, Data, 90

Header가 없는 경우 201, AI, 85

202, Data, 90

203, AI, 78

- header가 없는 파일을 그냥 읽으면 첫 번째 데이터 행이 column name 자리를 차지하게 된다.
- 이때는 header=None으로 첫 행도 데이터임을 알리고, names=[...]로 column name을 직접 지정한다.

Topic 4. Metadata Rows Before the Header

공공데이터나 기관 파일에는 실제 표 위에 출처나 설명 문장이 몇 줄 붙어 있는 경우가 많다.

```
Introduction to Data Analysis Score
Generated: 2026-03-31
ID,Major,Score
201,AI,85
202,Data,90
```

- 첫 두 줄은 데이터가 아니라 파일에 대한 설명이다.
- 실제 header가 몇 번째 줄에 있는지 먼저 확인한다.
- `skiprows`에 건너뛴 줄 수를 지정하면 앞의 설명 행을 무시하고 읽는다.

파일을 Excel이나 메모장으로 직접 열어 구조를 눈으로 확인하는 것도 좋은 방법이다. 코드만으로 모든 것을 추측할 필요는 없다.

Topic 5. Character Encoding

컴퓨터는 문자를 그대로 저장하지 않고 숫자로 변환해서 저장한다. **Encoding**은 어떤 문자를 어떤 숫자로 바꿀지 정해 놓은 규칙이다.

Encoding	특징	흔한 상황
UTF-8	국제적으로 널리 사용	Colab, web, modern system
CP949	한국어 Windows에서 많이 사용	오래된 CSV, 기관 파일

- 저장할 때와 다른 encoding으로 읽으면 `UnicodeDecodeError`가 발생하거나 한글이 깨진다.
- 오류 없이 열리더라도 한글이 깨진 문자로 표시될 수 있다.

Topic 5. Encoding Is Not a Language Option

Encoding은 “한국어 파일이면 한국어 encoding을 쓴다”는 식의 언어 선택 옵션이 아니다.

- 같은 한글 문장이라도 UTF-8과 CP949로 저장하면 파일 내부의 숫자 배열이 달라진다.
- 파일을 저장할 때 쓴 **encoding**과 같은 것을 읽을 때도 지정해야 글자가 정상으로 나온다.
- 어떤 encoding인지 모를 때는 파일을 만든 프로그램이나 제공 기관의 안내를 먼저 확인한다.

encoding 오류가 난다고 값을 무작위로 바꿔 가며 시도하기보다, **파일을 어디서 어떻게 만들었는지 먼저 확인**하는 것이 빠르다.

Topic 6. Important Loading Options

`pd.read_csv()`에서 자주 쓰는 option을 정리하면 다음과 같다.

Option	의미
<code>sep</code>	열 사이의 구분 문자 지정
<code>header</code>	<code>column name</code> 으로 쓸 행 번호 지정
<code>names</code>	<code>column name</code> 을 직접 지정
<code>encoding</code>	문자 <code>encoding</code> 방식 지정 (UTF-8, CP949 등)
<code>usecols</code>	읽어 올 <code>column</code> 을 골라서 지정
<code>dtype</code>	특정 <code>column</code> 의 <code>type</code> 을 강제로 지정
<code>parse_dates</code>	날짜 <code>type</code> 으로 읽을 <code>column</code> 지정
<code>na_values</code>	결측으로 처리할 값 지정 (예: <code>unknown</code> , <code>-99</code>)
<code>skiprows</code>	앞에서 건너될 행 수 지정

Topic 6. Read Only What You Need

열이 많은 파일에서는 필요한 column만 골라서 읽는 것이 효율적이다.

ID	Date	Store	Product	Qty	Revenue
1	03-01	A	Coffee	20	80000
2	03-02	A	Tea	15	45000
3	03-03	B	Coffee	25	100000

예를 들어 매출 분석에 ID, Date, Store, Revenue만 필요하다면 Product와 Qty는 처음부터 읽지 않아도 된다.

`usecols`는 편의 기능이기도 하지만, 큰 dataset에서는 메모리 사용량과 loading 시간을 크게 줄여 주는 중요한 option이다.

Topic 6. Specify Types When Necessary

pandas는 각 열의 type을 자동으로 추론하지만, 항상 우리가 원하는 방식으로 읽히는 것은 아니다.

- 우편번호 01234가 숫자로 읽히면 앞의 0이 사라져 1234가 된다. (string으로 읽어야 함)
- ID처럼 숫자이지만 계산에 쓰지 않는 열은 string(문자열)로 읽어 두는 것이 안전하다.
- 날짜 열은 parse_dates를 지정하면 파일을 읽는 단계에서 바로 datetime type으로 가져올 수 있다.
- 금액 열에 쉼표(1,000)나 통화기호(\)가 섞여 있으면 숫자로 자동 인식되지 않고 string으로 인식.(thousands=','등의 옵션 필요, 통화기호는 문자열 치환 후 숫자로)

자동 추론은 편리하지만, ID · 날짜 · 코드처럼 **type**이 중요한 열은 직접 지정하는 습관을 들이는 것이 좋다.

Topic 7. Validate Immediately After Loading

파일을 불러오면 바로 다음 여섯 가지를 확인한다.

1. **크기**: 예상한 행 수와 열 수인가?
2. **Column name**: 이름이 제대로 들어왔는가?
3. **데이터 일부**: 값이 올바른 열에 들어가 있는가?
4. **Type**: 숫자는 숫자로, 날짜는 날짜로 읽혔는가?
5. **결측**: 읽는 과정에서 예상치 못한 결측이 생기지 않았는가?
6. **값 범위**: 점수, 금액, 날짜가 말이 되는 범위 안에 있는가?

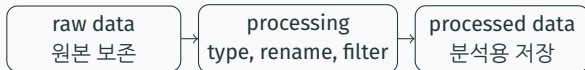
이 점검은 본격적인 분석을 시작하기 전 데이터가 제대로 읽혔는지 확인하는 단계이다. 이 과정을 건너뛰면 잘못 읽힌 데이터를 그대로 분석하는 실수가 생긴다.

Topic 7. A Simple Import Checklist

Check	Python	확인 질문
크기 데이터 일부	<code>df.shape</code> <code>df.head()</code>	행과 열 수가 예상과 같은가? 값이 엉뚱한 열에 들어가 있지 않은가?
Type 결측	<code>df.info()</code> <code>df.isna().sum()</code>	숫자는 숫자, 날짜는 날짜 <code>type</code> 인가? 읽는 과정에서 결측이 생기지 않았는가?
값 범위 범주	<code>df.describe()</code> <code>value_counts()</code>	말이 안 되는 값이 없는가? 오타나 예상하지 못한 <code>category</code> 값이 있는가?

Topic 8. Raw and Processed Data

원본 파일과 처리 결과를 같은 파일에 덮어쓰지 않도록 한다.



- raw file은 손대지 않고 그대로 보관한다. 나중에 처음부터 다시 작업할 수 있는 보험이 된다.
- 처리된 결과는 다른 이름이나 다른 folder에 저장한다.
- 어떤 코드로 어떤 파일을 만들었는지 남겨 두면 분석 과정을 나중에 그대로 재현할 수 있다.

Topic 8. A Reproducible Folder Structure

Folder	역할
raw/	원본 file, 수정하지 않음
processed/	정리된 분석용 file
code/	notebook 또는 Python script
output/	table, figure, report

예를 들어 다음과 같이 저장할 수 있다.

```
raw/shuttle_usage.csv
processed/shuttle_usage_clean.csv
code/week3_analysis.ipynb
```

파일 이름에 `final`, `final2`, `real_final`을 반복해서 붙이기보다, `folder`와 버전 규칙을 미리 정해 두는 편이 낫다.

Diagnosing File Loading Problems

파일이 제대로 읽히지 않을 때는 화면에 나타나는 증상을 보고 원인을 찾는다.

증상	가능 원인	먼저 확인할 것
파일을 찾을 수 없음	path가 잘못됨	working directory와 파일 위치 확인
여러 열이 한 열로 뭉쳐서 읽힘	delimiter가 다름	sep 확인
첫 데이터 행이 열 이름이 됨	header가 없는 파일	header=None 지정
한글이 깨지거나 오류 발생	encoding이 맞지 않음	encoding option 확인
날짜 열이 문자로 읽힘	type 자동 추론 실패	parse_dates 지정
열 수가 예상보다 적음	header 또는 delimiter 문제	header와 sep 함께 확인

Week 3-1 Summary

- 데이터의 출처와 수집 이력을 기록해 두면 나중에 같은 파일의 의미를 다시 확인할 수 있다.
- TXT, CSV, Excel은 저장 구조가 다르므로 format에 맞는 읽기 함수를 써야 한다.
- working directory와 relative path를 이해하면 파일을 찾지 못하는 오류를 줄일 수 있다.
- delimiter나 header 설정이 잘못되면 표의 구조 자체가 어긋난다.
- encoding은 문자를 숫자로 바꾸는 규칙이다. 저장할 때와 읽을 때 같은 encoding을 써야 한글이 깨지지 않는다.
- 파일을 불러온 직후에는 행 · 열 수, column name, type, 결측, 값 범위를 바로 확인한다.
- raw data와 processed data를 분리해서 저장해야 분석 과정을 나중에 처음부터 재현할 수 있다.

Week 3-2. Python Practice

Week 3-2 Overview

- 현재 working directory와 그 안의 파일 목록을 확인한다.
- CSV를 직접 만들어 저장한 뒤, option을 바꿔 가며 다시 읽어 본다.
- delimiter, header, encoding, type, 날짜 관련 문제를 코드로 직접 해결한다.
- TXT와 Excel file을 읽고 저장한다.
- 마지막에는 raw file을 불러와 처리하고 저장하는 전체 흐름을 하나의 코드로 연결한다.

코드가 오류 없이 실행됐다고 끝이 아니다. 불러온 결과가 원본과 같은지 매 단계마다 직접 출력해서 확인한다.

Setup: Libraries

파일 경로와 폴더를 다루는 표준 라이브러리

```
import os  
from pathlib import Path
```

표 형태 데이터를 읽고 처리하는 라이브러리

```
import pandas as pd
```

결측값(NaN) 표현시 필요함

```
import numpy as np
```

- os: 현재 folder와 file 목록 확인
- Path: 문자열 대신 구조적으로 경로를 다룸
- pandas: table file 읽기와 쓰기
- numpy: 수치 계산, 통계계산, 난수, 결측값처리

Practice 1. Checking the Working Directory

Python이 현재 어느 폴더를 기준으로 파일을 찾는지 출력한다

print(os.getcwd())

현재 폴더 안에 있는 파일과 폴더 목록을 출력한다

print(os.listdir())

파일을 찾지 못하는 오류가 날 때 가장 먼저 확인할 두 가지이다.

Colab에서는 notebook 실행 위치와 Google Drive 폴더 위치가 다를 수 있으니 주의한다.

Practice 1. Building Paths with Pathlib

"raw"라는 이름의 폴더 경로를 Path 객체로 만든다

`raw_dir = Path("raw")`

"processed"라는 이름의 폴더 경로를 Path 객체로 만든다

`processed_dir = Path("processed")`

폴더가 없으면 새로 만들고, 이미 있으면 오류 없이 넘어간다

`raw_dir.mkdir(exist_ok=True)`

`processed_dir.mkdir(exist_ok=True)`

/ 기호로 폴더와 파일 이름을 연결하여 전체 경로를 만든다

`csv_path = raw_dir / "sales.csv"`

`print(csv_path)`

"sales.csv"

`exist_ok=True`는 이미 폴더가 있어도 오류를 발생시키지 않는다.

Practice 2. Creating a Sample Dataset

```
# 딕셔너리로 열을 정의하고 DataFrame을 만든다
# 각 키(key)가 열 이름이 되고, 값의 리스트가 그 열의 데이터가 된다
sales = pd.DataFrame({
    "Date": ["2026-03-01", "2026-03-02", "2026-03-03",
            "2026-03-04", "2026-03-05", "2026-03-06"],
    "Store": ["A", "A", "B", "B", "C", "C"],
    "Product": ["Coffee", "Tea", "Coffee",
               "Cake", "Tea", "Coffee"],
    "Quantity": [20, 15, 25, 8, 18, 30],
    "Revenue": [80000, 45000, 100000, 56000, 54000, 120000]
})

print(sales)
```

Handwritten red notes:

- A circle around the opening curly brace of the dictionary.
- A circle around the closing curly brace of the dictionary.
- A circle around the "Date" key.
- A circle around the "Store" key.
- A circle around the "Product" key.
- A circle around the "Quantity" key.
- A circle around the "Revenue" key.
- A circle around the closing curly brace of the DataFrame constructor.
- A circle around the "print(sales)" statement.

Practice 2. Saving a CSV File

```
# DataFrame을 CSV 파일로 저장한다
# index=False: 행 번호(0,1,2...)를 파일에 저장하지 않는다
# encoding="utf-8": 문자 저장 규칙을 UTF-8로 지정한다
sales.to_csv(csv_path, index=False, encoding="utf-8")

# 지정한 경로에 파일이 실제로 생성되었는지 확인한다 (True/False)
print(csv_path.exists())

# 저장된 파일의 크기를 byte 단위로 확인한다
print(csv_path.stat().st_size)
```

0
1
2
3

index=False를 생략하면 0, 1, 2... 같은 행 번호가 첫 번째 열로 저장되어 나중에 읽을 때 불필요한 열이 생긴다.

Practice 2. Reading the CSV File

저장한 CSV 파일을 다시 DataFrame으로 불러온다

```
loaded = pd.read_csv(csv_path)
```

첫 5행을 출력하여 값이 열에 맞게 읽혔는지 눈으로 확인한다

```
print(loaded.head())
```

(행 수, 열 수)를 출력한다. 원본과 같은지 확인한다

```
print(loaded.shape)
```

열 이름 목록을 출력한다. 예상한 이름과 같은지 확인한다

```
print(loaded.columns)
```

6행 5열로 읽혔는지, column name이 예상한 것과 같은지 확인한다.

Practice 2. Checking Types after Loading

행 수, 열 이름, 각 열의 타입, 결측 개수를 한 번에 출력한다

```
print(loaded.info())
```

각 열의 저장 타입만 따로 출력한다

```
print(loaded.dtypes)
```

Date 열은 날짜이지만 CSV 파일 안에서는 텍스트로 저장된다. 그냥 읽으면 object(문자) type으로 표시되어 날짜 계산을 할 수 없다. 날짜 계산이 필요하다면 `parse_dates`로 지정해야 한다.

Practice 3. Reading a Semicolon File

```
semicolon_path = raw_dir / "sales_semicolon.csv"

# 열 구분자를 semicolon(;)으로 바꿔 저장한다
sales.to_csv(semicolon_path, index=False, sep=";")

# 구분자를 지정하지 않고 기본값(comma)으로 읽어 본다
# semicolon이 comma로 인식되지 않아 전체가 1개 열로 묶인다
wrong = pd.read_csv(semicolon_path)
print(wrong.head())
print(wrong.shape)
```

원래 5개 열인데 1개 열로 읽었다면 delimiter가 잘못 지정된 것이다.

Practice 3. Fixing the Delimiter

파일에서 실제로 사용한 구분자(;)를 sep에 정확히 지정한다

```
correct = pd.read_csv(semicololon_path, sep=";")
```

이번에는 6행 5열로 정상적으로 읽혔는지 확인한다

```
print(correct.head())
```

```
print(correct.shape)
```

sep 하나만 올바르게 지정해도 표의 구조 전체가 복구된다. option 하나가 결과를 완전히 바꿀 수 있다.

Practice 4. Reading a File without a Header

```
no_header_path = raw_dir / "score_no_header.csv"
```

```
# 열 이름 없이 데이터만 있는 CSV 내용을 직접 작성한다
```

```
raw = ""201,AI,85
```

```
202,Data,90
```

```
203,AI,78
```

```
""
```

```
# 작성한 내용을 파일로 저장한다
```

```
no_header_path.write_text(raw, encoding="utf-8")
```

```
# header 옵션을 지정하지 않으면 첫 행을 열 이름으로 읽는다
```

```
# 그 결과 201번 학생 데이터가 열 이름 자리를 차지하게 된다
```

```
wrong = pd.read_csv(no_header_path)
```

```
print(wrong)
```

첫 번째 학생의 정보가 column name 자리로 올라가고, 실제 데이터 행이 한 줄 줄어든다.

Practice 4. Supplying Column Names

```
score = pd.read_csv(  
    no_header_path,  
    # header=None: 첫 줄도 데이터이므로 열 이름으로 쓰지 않는다  
    header=None,  
    # names: 열 이름을 우리가 직접 지정한다  
    names=["ID", "Major", "Score"]  
)  
  
print(score)  
print(score.shape)
```

header=None과 names를 함께 쓰면 header가 없는 파일을 정확하게 읽을 수 있다.

Practice 4. Skipping Metadata Rows

```
meta_path = raw_dir / "score_with_meta.csv"
```

```
# 실제 데이터 위에 설명 텍스트가 2줄 있는 파일을 만든다
```

```
raw = """Sejong University Learning Data
```

```
Generated: 2026-03-31
```

```
ID, Major, Score
```

```
201, AI, 85
```

```
202, Data, 90
```

```
"""
```

```
meta_path.write_text(raw, encoding="utf-8")
```

```
# skiprows=2: 앞의 2줄(설명 텍스트)을 건너뛰고 3번째 줄부터 읽는다
```

```
score_meta = pd.read_csv(meta_path, skiprows=2)
```

```
print(score_meta)
```

실제 header가 몇 번째 줄인지 파일을 직접 열어 확인한 뒤, 그 위의 줄 수를 skiprows에 지정한다.

Practice 5. UTF-8 and CP949

Latex

한글이 포함된 DataFrame을 만든다

```
korean = pd.DataFrame({  
    "Region": [r"\uC119\uC6B8", r"\uB080\uC0B0", r"\uB300uC804"],  
    "Sales": [120, 95, 88]  
})
```

cp949_path = raw_dir / "korean_cp949.csv"

한국어 Windows에서 생성된 파일을 가정하여 CP949로 저장한다

```
korean.to_csv(cp949_path, index=False, encoding="cp949")
```

저장할 때 쓴 encoding(cp949)과 같은 방식으로 읽어야 한글이 깨지지 않는다

```
korean_loaded = pd.read_csv(cp949_path, encoding="cp949")
```

```
print(korean_loaded)
```

utf-8

Practice 5. What Happens with the Wrong Encoding?

try-except

```
# CP949로 저장된 파일을 UTF-8로 읽으면 오류가 발생한다
# try-except로 오류를 잡아서 내용을 출력하고 프로그램을 계속 실행한다
try:
    wrong_encoding = pd.read_csv(cp949_path, encoding="utf-8")
    print(wrong_encoding)
except UnicodeDecodeError as e:
    # 어떤 encoding 오류인지 메시지를 출력한다
    print("Encoding error:", e)
```

일부러 오류를 발생시켜 보면 encoding 문제가 다른 오류와 어떻게 다른지 구분할 수 있다. 오류 메시지를 읽는 것이 문제 해결의 시작이다.

Practice 6. Selecting Columns while Loading

usecols: 파일 전체를 읽지 않고 필요한 열만 골라서 읽는다

여기서 5개 열 중 3개만 불러온다

```
selected = pd.read_csv(  
    csv_path,  
    usecols=["Date", "Store", "Revenue"]  
)
```

3개 열만 읽었는지 shape으로 확인한다 (6행 3열이어야 한다)

```
print(selected.head())
```

```
print(selected.shape)
```

ID나 우편번호처럼 앞에 0이 중요한 값은 반드시 dtype으로 string type을 지정해야 값이 바뀌지 않는다.

Practice 6. Parsing Dates while Loading

```
#parse_dates: 기정한 열을 문자가 아니라 날짜 데이터로 읽는다
sales_date = pd.read_csv(csv_path, parse_dates=["Date"])

#Date 열이 object(문자)가 아니라 datetime64로 읽혔는지 확인한다
print(sales_date.dtypes)

#날짜 타입이 되면 .dt 접근자로 연도, 월, 요일 등을 바로 꺼낼 수 있다
sales_date["Month"] = sales_date["Date"].dt.month
sales_date["Weekday"] = sales_date["Date"].dt.day_name()

print(sales_date[["Date", "Month", "Weekday"]].head())
```

Practice 6. Setting a Specific dtype

```
id_path = raw_dir / "customer_id.csv"
```

```
# 앞에 0이 붙은 ID가 포함된 파일을 만든다
```

```
raw = """ID,Amount
```

```
001,12000
```

```
002,18500
```

```
010,9000
```

```
"""
```

```
id_path.write_text(raw, encoding="utf-8")
```

```
# dtype="ID": "string": ID 열을 숫자가 아니라 문자로 읽도록 강제한다
```

```
# 숫자로 읽으면 001이 1이 되어 앞의 0이 사라진다
```

```
customer = pd.read_csv(id_path, dtype={"ID": "string"})
```

```
print(customer)
```

```
print(customer.dtypes)
```

{ID: "string"}

ID나 우편번호처럼 앞에 0이 중요한 값은 반드시 dtype으로 string type을 지정한다.

Practice 6. Defining Missing Tokens at Import

```
missing_path = raw_dir / "study_time.csv"
```

```
# 결측을 뜻하는 값이 두 가지 형태로 섞인 파일을 만든다
```

```
raw = """ID,StudyTime
```

```
1,120
```

```
2,unknown
```

```
3,-99
```

```
4,90
```

```
"""
```

```
missing_path.write_text(raw, encoding="utf-8")
```

```
# na_values: 파일에서 결측을 뜻하는 값들을 지정한다
```

```
# "unknown"과 "-99"를 읽을 때 자동으로 NaN(결측)으로 바꾼다
```

```
study = pd.read_csv(
```

```
    missing_path,
```

```
    na_values=["unknown", "-99"]
```

```
)
```

```
print(study)
```

```
# 열별로 NaN이 몇 개 생겼는지 확인한다
```

```
print(study.isna().sum())
```

Handwritten notes in red ink:

- A large circle around "NaN" with a cross through it.
- A circle around "NA" with "null" written below it.
- A circle around "unknown" with "1" written below it.

na_values로 결측 표현을 파일을 읽는 단계에서 처리하면, 나중에 따로 replace를 실행하지 않아도 된다.

Practice 7. Reading and Writing Plain TXT

```
memo_path = raw_dir / "memo.txt"

# "w": 쓰기 모드로 파일을 연다. 파일이 없으면 새로 만들고 있으면 덮어쓴다
with open(memo_path, "w", encoding="utf-8") as f:
    f.write("Data analysis starts with data.\n")
    f.write("File structure must be checked first.\n")

# "r": 읽기 모드로 파일을 연다
with open(memo_path, "r", encoding="utf-8") as f:
    # 파일 전체 내용을 하나의 문자열로 읽어 text 변수에 저장한다
    text = f.read()

print(text)
```

Plain TXT는 표 형식이 아닌 연속된 텍스트이므로 DataFrame으로 자동 변환되지 않는다.

Practice 7. Reading a Delimited TXT as a Table

```
txt_path = raw_dir / "score.txt"
```

```
# 값 사이를 공백으로 구분한 텍스트 파일을 만든다
```

```
raw = """ID Score Age
```

```
1 85 20
```

```
2 90 21
```

```
3 78 19
```

```
"""
```

```
txt_path.write_text(raw, encoding="utf-8")
```

```
# sep=r" " 하나 이상의 공백을 일 구분자로 사용한다
```

```
# 여러 칸 띄어쓰기가 섞여 있어도 올바르게 나눈다
```

```
score_txt = pd.read_csv(txt_path, sep=r"\s+")
```

```
print(score_txt)
```

1 85 20
2 90 21
3 78 19

sep=r"\s+"는 공백이 한 칸이든 여러 칸이든 모두 delimiter로 처리한다. 공백 하나만 지정하는 것보다 안전하다.

Practice 7. Saving and Reading Excel

```
excel_path = raw_dir / "sales.xlsx"

#DataFrame을 Excel 파일로 저장한다
#sheet_name: 저장할 시트의 이름을 "March"로 지정한다
#index=False: 행 번호를 별도 열로 저장하지 않는다
sales.to_excel(excel_path, sheet_name="March", index=False)

#저장한 Excel 파일을 다시 읽는다
#sheet_name: 어느 시트를 읽을지 이름으로 지정한다
sales_excel = pd.read_excel(excel_path, sheet_name="March")

print(sales_excel.head())
```

환경에 따라 Excel 파일을 읽고 쓰려면 `openpyxl` 패키지가 별도로 필요할 수 있다.

Practice 7. Writing Multiple Excel Sheets

```
multi_path = raw_dir / "sales_multi.xlsx"
```

```
# 상점별 매출 합계를 구해 요약 DataFrame을 만든다
```

```
summary = sales.groupby("Store")["Revenue"].sum().reset_index()
```

```
# ExcelWriter: 하나의 파일에 여러 시트를 쓸 때 사용한다
```

```
with pd.ExcelWriter(multi_path) as writer:
```

```
# "Raw" 시트에 원본 데이터를 저장한다
```

```
sales.to_excel(writer, sheet_name="Raw", index=False)
```

```
# "Summary" 시트에 요약 결과를 저장한다
```

```
summary.to_excel(writer, sheet_name="Summary", index=False)
```

```
# 파일 안에 시트가 몇 개 있는지, 이름이 맞는지 확인한다
```

```
print(pd.ExcelFile(multi_path).sheet_names)
```

o
r
i
g
i
n
a
l

Practice 7. Reading All Excel Sheets

```
# sheet_name=None: 모든 시트를 한 번에 읽는다
# 결과는 딕셔너리 형태로 반환된다. 키=시트 이름, 값=DataFrame
all_sheets = pd.read_excel(multi_path, sheet_name=None)

# 읽어들인 시트 이름 목록을 출력한다
print(all_sheets.keys())

# "Raw" 시트의 앞 5행을 출력한다
print(all_sheets["Raw"].head())

# "Summary" 시트 전체를 출력한다
print(all_sheets["Summary"])
```

결과가 딕셔너리 형태이므로 시트 이름을 key로 지정하면 해당 DataFrame을 바로 꺼낼 수 있다.

Practice 8. Basic Processing after Loading

#Date 열을 날짜 타입으로 읽어온다

```
data = pd.read_csv(csv_path, parse_dates=["Date"])
```

columns="Quantity": "Qty": 열 이름 Quantity를 Qty로 바꾼다

```
data = data.rename(columns={"Quantity": "Qty"})
```

Revenue를 Qty로 나눠 개당 단가를 계산하고 새 열로 추가한다

```
data["UnitPrice"] = data["Revenue"] / data["Qty"]
```

```
print(data.head())
```

파일을 불러온 뒤의 가공 작업은 raw 파일을 수정하는 것이 아니라 DataFrame 안에서만 이루어진다.

Practice 8. Filtering and Grouping

```
# Revenue가 80,000 이상인 행만 선택한다
high_revenue = data[data["Revenue"] >= 80000]
print(high_revenue)

# Store를 기준으로 그룹을 나누고 각 그룹의 매출을 요약한다
# n: 해당 Store의 거래 건수
# total_revenue: 매출 합계, mean_revenue: 평균 매출
store_summary = data.groupby("Store").agg(
    n=("Revenue", "count"),
    total_revenue=("Revenue", "sum"),
    mean_revenue=("Revenue", "mean")
)

print(store_summary)
```

[0, 0]

Practice 8. Saving Processed Data

```
#processed 폴더 안에 저장할 파일 경로를 만든다
processed_csv = processed_dir / "sales_processed.csv"
processed_xlsx = processed_dir / "sales_processed.xlsx"

#처리된 DataFrame을 CSV로 저장한다
data.to_csv(processed_csv, index=False, encoding="utf-8")

#동일한 내용을 Excel로도 저장한다
data.to_excel(processed_xlsx, index=False)

#두 파일이 실제로 생성되었는지 확인한다 (True여야 한다)
print(processed_csv.exists())
print(processed_xlsx.exists())
```

Practice 8. Round-Trip Validation

```
# 저장한 processed CSV를 다시 불러온다
# Date 열을 날짜 타입으로 지정하여 저장 전과 타입이 같게 한다
check = pd.read_csv(processed_csv, parse_dates=["Date"])

# 다시 읽은 결과의 행/열 수와 앞 5행을 출력한다
print(check.shape)
print(check.dtypes)
print(check.head())

# 저장 전 DataFrame과 다시 읽은 결과의 행/열 수가 같은지 비교한다
# True이면 저장 과정에서 데이터 손실이 없었다는 뜻이다
print(data.shape == check.shape)
```

저장이 됐다는 것만 믿지 않고, 다시 불러와서 구조가 그대로인지 직접 확인한다.

Complete Workflow: Import

```
source = raw_dir / "sales.csv"

#필요한 열, 날짜 타입을 한 번에 지정하여 파일을 읽는다
df = pd.read_csv(
    source,
    #5개 열 중 필요한 5개를 명시적으로 지정한다
    usecols=["Date", "Store", "Product", "Quantity", "Revenue"],
    #Date 열을 문자가 아닌 날짜 타입으로 읽는다
    parse_dates=["Date"]
)

#읽은 직후 행/열 수, 앞부분, 타입을 모두 확인한다
print(df.shape)
print(df.head())
print(df.info())
```

파일을 읽은 직후 구조를 점검하는 것이 모든 분석의 첫 번째 단계이다.

Complete Workflow: Process and Save

```
# 열 이름을 더 짧게 바꾼다
df = df.rename(columns={"Quantity": "Qty"})

# 개당 단가를 계산하여 새 열로 추가한다
df["UnitPrice"] = df["Revenue"] / df["Qty"]

# Revenue가 50,000 이상인 행만 남기고 별도 DataFrame으로 저장한다
# .copy()를 붙여 원본 df와 완전히 분리된 새 객체를 만든다
result = df[df["Revenue"] >= 50000].copy()

target = processed_dir / "sales_analysis.csv"
# 처리된 결과를 processed 폴더에 저장한다
result.to_csv(target, index=False, encoding="utf-8")

# 저장 후 다시 읽어서 행/열 수가 유지되었는지 최종 확인한다
check = pd.read_csv(target, parse_dates=["Date"])
print(result.shape, check.shape)
print(check.head())
```

From File to Analysis-Ready Data

단계	작업	확인할 내용
1	출처와 파일 확인	어디서 어떻게 얻은 파일인가
2	path 확인	파일이 실제로 그 위치에 있는가
3	format 확인	CSV, TXT, Excel 중 무엇인가
4	읽기 option 지정	delimiter, header, encoding
5	loading	필요한 열과 type을 정확히 읽었는가
6	결과 점검	크기 · column name · type · 결측 · 값 범위
7	데이터 처리	column 이름 변경, 새 열 추가, filter, group
8	저장	raw와 processed를 별도 파일로 저장했는가
9	재확인	저장한 파일을 다시 불러왔을 때 구조가 유지되는가

Checklist for Week 3

- 데이터 출처와 수집 시점을 기록할 수 있는가?
- TXT, CSV, Excel의 차이를 설명할 수 있는가?
- working directory와 relative path의 의미를 설명할 수 있는가?
- delimiter가 잘못 지정되었을 때 나타나는 증상을 알아볼 수 있는가?
- header가 없는 파일을 올바르게 읽을 수 있는가?
- UTF-8과 CP949의 차이가 언어 선택이 아니라 encoding 규칙의 차이라는 것을 아는가?
- usecols, dtype, parse_dates, na_values를 적절히 사용할 수 있는가?
- 파일을 불러온 직후 크기, column name, type, 결측을 확인하는가?
- raw file과 processed file을 분리해서 저장하는가?

Week 3 Summary

- 파일 불러오기는 원본 데이터의 구조와 의미를 Python DataFrame으로 정확히 옮기는 과정이다.
- path, delimiter, header, encoding은 파일을 읽은 결과를 직접 바꾸는 핵심 설정이다.
- pandas의 읽기 option을 활용하면 필요한 열, type, 날짜, 결측 표현을 파일을 읽는 단계에서 한 번에 지정할 수 있다.
- TXT는 내부가 문장인지 구분된 표인지 먼저 확인하고, Excel은 어느 sheet에 데이터가 있는지 확인한다.
- 파일을 불러온 직후에는 크기, 데이터 일부, type, 결측을 점검한다.
- 원본은 raw folder에 그대로 보존하고, 처리 결과는 processed file로 별도 저장한다.
- 저장한 파일은 다시 불러와 행·열 수가 그대로인지 확인한다.

Review Exercise

대중교통 이용 데이터를 가정하여 다음 작업을 수행하시오.

1. Date, Route, Passengers, Revenue를 포함하고 최소 8개의 observation을 갖는 DataFrame을 만드시오.
2. 이를 shuttle.csv로 저장하되 row index는 저장하지 마시오.
3. 저장한 file을 다시 불러와 shape, column name, data type을 확인하시오.
4. Date를 datetime으로 읽고 Route, Passengers, Revenue만 선택하여 불러오시오.
5. Revenue가 50,000 이상인 row만 선택하시오.
6. Route별 passenger 수와 revenue의 합계 및 평균을 계산하시오.
7. processed 결과를 별도의 CSV와 Excel file로 저장하시오.
8. 저장한 CSV를 다시 읽어 원래 processed DataFrame과 행과 열 수가 같은지 확인하시오.

Review Exercise: 코드 정답 예시 1

1-2. DataFrame 생성과 CSV 저장

```
import pandas as pd
from pathlib import Path

# 8개의 관측치를 가진 대중교통 이용 DataFrame을 만든다
shuttle = pd.DataFrame({
    "Date": ["2026-03-02", "2026-03-02", "2026-03-03",
            "2026-03-03", "2026-03-04", "2026-03-04",
            "2026-03-05", "2026-03-05"],
    "Route": ["A", "B", "A", "B", "A", "B", "A", "B"],
    "Passengers": [42, 31, 48, 35, 51, 33, 46, 39],
    "Revenue": [63000, 46500, 72000, 52500,
               76500, 49500, 69000, 58500]
})

# 행 번호 없이 CSV로 저장한다
path = Path("shuttle.csv")
shuttle.to_csv(path, index=False, encoding="utf-8")

print(shuttle)
```

Review Exercise: 코드 정답 예시 2

3. 저장한 file 다시 불러오기와 구조 확인

```
# 저장한 파일을 기본값으로 불러온다
loaded = pd.read_csv("shuttle.csv")

# 8행 4열인지 확인한다
print(loaded.shape)

# 열 이름이 Date, Route, Passengers, Revenue인지 확인한다
print(loaded.columns)

# 각 열의 저장 타입을 확인한다 (Date는 아직 object일 것이다)
print(loaded.dtypes)

# 앞 5행을 출력하여 값이 올바르게 읽혔는지 눈으로 확인한다
print(loaded.head())
```

Review Exercise: 코드 정답 예시 3

4. Date type과 필요한 column 지정

```
selected = pd.read_csv(  
    "shuttle.csv",  
    #Date, Route, Passengers, Revenue 4개 열만 읽는다  
    usecols=["Date", "Route", "Passengers", "Revenue"],  
    #Date 열을 문자가 아닌 날짜 타입으로 읽는다  
    parse_dates=["Date"]  
)  
  
#Date가 datetime64 타입으로 읽혔는지 확인한다  
print(selected.dtypes)  
print(selected.head())
```

Date가 datetime64 타입으로 읽혔는지 확인한다.

Review Exercise: 코드 정답 예시 4

5. Revenue 기준 filtering

```
# Revenue가 50,000 이상인 행만 선택한다
# .copy()로 원본 selected와 분리된 새 DataFrame을 만든다
high_revenue = selected[
    selected["Revenue"] >= 50000
].copy()

print(high_revenue)

# 조건을 만족한 행의 수를 확인한다
print(high_revenue.shape)
```

.copy()를 붙이지 않으면 원본과 연결된 뷰가 생겨 나중에 수정할 때 경고가 발생할 수 있다.

Review Exercise: 코드 정답 예시 5

6. Route별 요약

```
# Route를 기준으로 그룹을 나눠 승객 수와 매출을 요약한다
route_summary = selected.groupby("Route").agg(
    # 각 노선의 운행 횟수
    passenger_sum = ("Passengers", "sum"),
    # 각 노선의 총 승객 수
    passenger_mean = ("Passengers", "mean"),
    # 각 노선의 총 매출
    revenue_sum = ("Revenue", "sum"),
    # 각 노선의 평균 매출
    revenue_mean = ("Revenue", "mean")
)

# 소수점 첫째 자리까지 출력한다
print(route_summary.round(1))
```

Review Exercise: 코드 정답 예시 6

7. Processed result 저장

```
# 처리된 결과를 원본과 다른 이름으로 csv에 저장한다
high_revenue.to_csv(
    "shuttle_processed.csv",
    index=False,
    encoding="utf-8"
)

# 동일한 결과를 Excel에도 저장한다
high_revenue.to_excel(
    "shuttle_processed.xlsx",
    index=False
)

# 저장 전 DataFrame의 크기를 확인해 둔다
print(high_revenue.shape)
```

Review Exercise: 코드 정답 예시 7

8. 저장 결과 다시 확인

```
# 저장한 processed CSV를 다시 불러온다
check = pd.read_csv(
    "shuttle_processed.csv",
    parse_dates=["Date"]
)

# 다시 읽은 결과의 크기와 앞 5행을 출력한다
print(check.shape)
print(check.head())

# 저장 전과 저장 후의 행/열 수가 같으면 True가 출력된다
same_shape = high_revenue.shape == check.shape
print(same_shape)
```

True가 출력되면 저장 전후 행·열 수가 같다는 의미다. 저장 과정에서 데이터 손실이 없었음을

Review Exercise: 결과 해석 예시

- 원본 데이터는 8개의 observation과 4개의 variable로 구성되어 있다.
- CSV를 다시 불러온 뒤 shape과 column name을 확인하여 표의 구조가 유지되었음을 확인하였다.
- Date를 datetime으로 읽어 이후 날짜별 분석에 바로 사용할 수 있는 형태로 만들었다.
- Revenue가 50,000 이상인 observation만 선택하여 별도의 processed DataFrame을 만들었다.
- Route별 승객 수와 revenue를 합계 및 평균으로 요약하여 노선 간 이용 규모를 비교할 수 있게 하였다.
- processed 결과를 CSV와 Excel로 별도 저장하여 raw file을 그대로 보존하였다.
- 저장한 CSV를 다시 읽은 뒤 shape을 비교하여 저장 과정에서 행과 열의 수가 변하지 않았음을 확인하였다.

본 교육콘텐츠는 교육부와 한국연구재단의 지원을 받아 제작한 AI융합
인재양성사업단의 콘텐츠입니다.